The Program “platemapper” is a Qiita compatible plugin based on Qiime2 and written in Python. Its main goal is to provide the User with an interactive 96 well plate layout for their biological research. It utilizes tools from the multi-omics data Science platform QIIME2 to convert a metadata spreadsheet of datasets into a visual output resembling a 96 well plate. The reason a visualized plate is advantageous would be to check your samples and plate for an increased risk of spill over. Spill over happens by applying samples of different characteristics (e.g *Wild Type* and *mutant*) side by side onto a plate, thus increasing the risk of possible spilling of one sample into the well of the other, rendering their results useless. In a perfect world, Scientists would not arrange and apply their samples by structure, they would randomly shuffle their samples and blindly apply them in their mixed order, thus mitigating spill over by defying a random approach to apply samples into wells. Another way to prevent spill over would be to leave enough distance between wells of different characteristics, if structuring the samples by taught conventions is absolutely necessary. Platemapper‘s function should be to interactively show the user their plates and immediately reveal if samples could be affected by spill over.

To accomplish this, platemapper uses mechanisms from scikit-bio to turn various datasets into a PCoA based ordination file which will then be utilized by a visualization tool called “Emperor” to display the Data as a 96 well plate layout. While officially using PCoA as the ordination type, platemapper does not use the usual features provided by PCoA, as no distances between paired objects will be measured. Also, multidimensional calculations of distances between pairs is not possible, as ordination coordinates and Emperors visual output are limited to 2D. Platemapper has relatively small requirements: it needs access to the QIIME2 plugin library and a file in which the metadata is provided, which needs to have 2 very specific columns which the user has to include: “well_id” and “plate_id”. It is necessary ,that these two columns are available, otherwise platemapper throw a controlled error. *well_id* being the coordinates of the 96 plate well, containing A-H on the x-axis and 1-12 on the y axis (*e.g. A1, C5, E11)*. Right now it is important to not reverse the X and Y axis, as platemapper is not yet capable of detecting if the Letter or number was written

first. *plate_id* differentiates the different plates which were used on this research if all of them show up in one single metadata file. If your research only contains one plate, there's still the need to include the column plate_id with the same value, as platemapper does not yet differentiate between one or many plates. As plates are already normally named on such metadata files, the user can easily rename the fitting column to *plate_id*.

The mechanism as how Platemapper turns a confusing looking table into a visual output resembling a 96 well plate begins with choosing/detecting a metadata file.

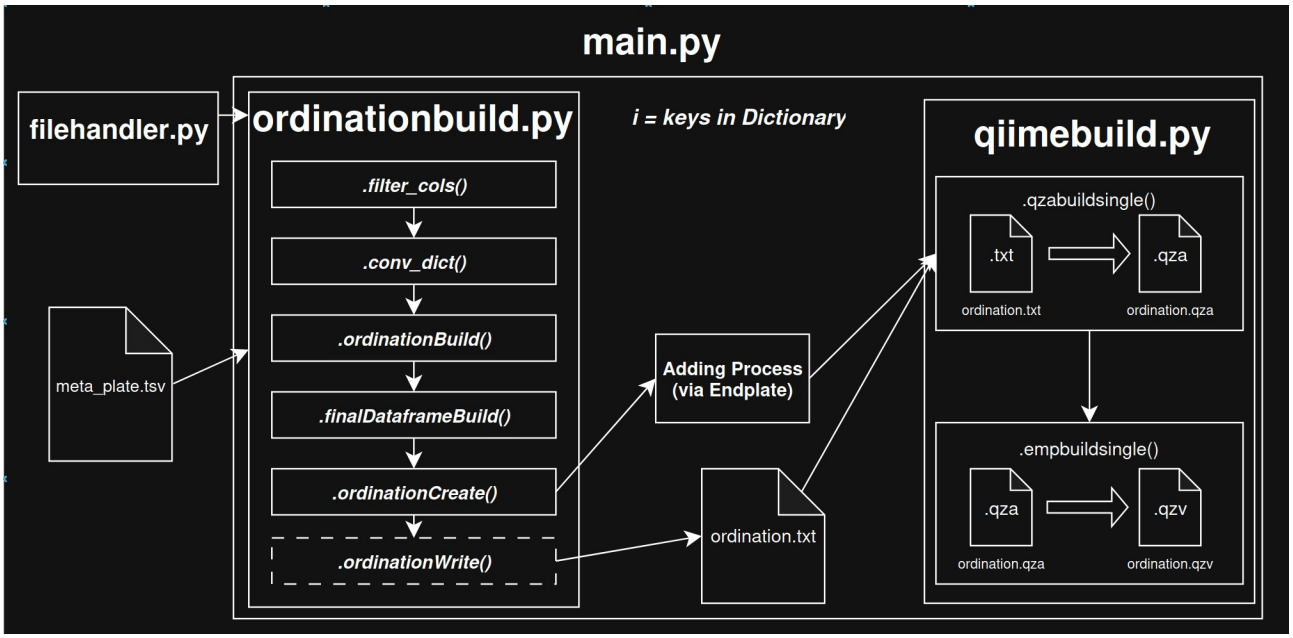

Figure 1: The entire functioning process of platemapper visualized and simplified

In the current version, the file still has to be chosen via a choose file window which is created via the python plugin *tkinter*. This will be erased in future versions, as the metadata file will be found in Qiita's standardized path. The file choosing function then reads the file (mostly a .tsv file) into a pandas DataFrame while also saving the path from which the file was chosen. After successful selection, platemapper creates the directories and Folders necessary to complete the conversion. This is handled by the file *filehandler.py* which creates following folder structure:

```
└── platemapper/
    ├── ...
    └── output/
        ├── artifact_results
        ├── emp_results
        └── ordination_results
```

After successfully creating the emperor output, which is located in *emp_results*, the files contained in *artifact_results* and *ordination_results* will get deleted as they are temporary files and not of use after getting a visual output.

After the folder structure is created and the File is chosen, platemapper will begin to alter the data in the DataFrame.. important to note: no original Data or Files which platemapper accesses will be modified or changed in the files themselves. The first step of creating the ordination is to filter the DataFrame by the column *plate_id*. platemapper takes all unique values of *plate_id* and groups all associated values based on *plate_id* values using the pandas function “groupby”. After this step, all separated and grouped plates will get saved into a Python dictionary where key shows the plate number and the associated value being the split DataFrame with all remaining values.

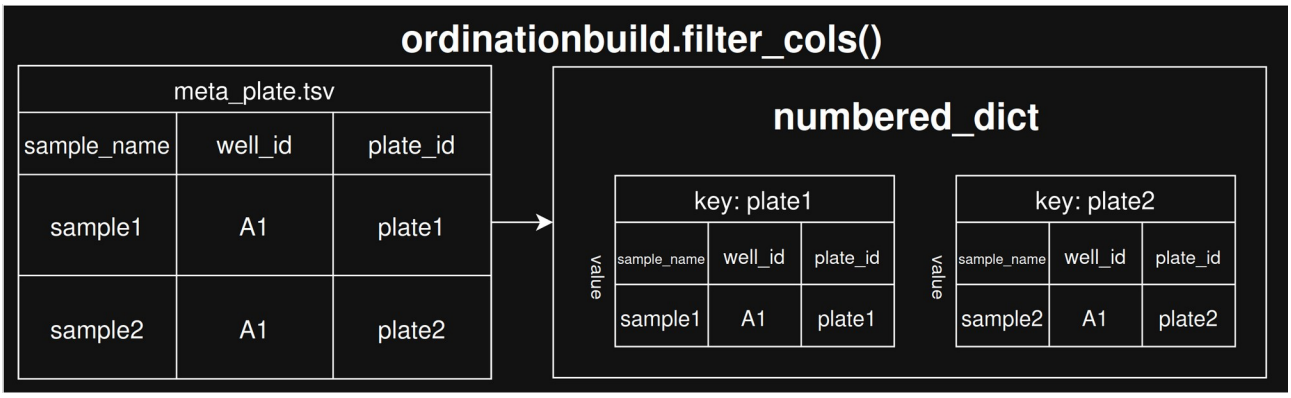

Figure 2: detailed visualization of function ordinationbuild.filter_cols()

In order to access the Keys in which the DataFrames of individual Plates are stored, platemapper simply iterates through the Dictionary and takes one DataFrame into a fresh variable. This step is necessary because the *OrdinationResults* function from scikit-bio does not seem to understand a DataFrame stored within a Dictionary. By taking the DataFrame out of the Dictionary and into a variable, scikit-bio gets a DataFrame within a normal variable. The Designated function *ordinationbuild.conv_dict()* takes the i’th key of the Dictionary and stores it in the variable *variabledf* for further use in ordination building.

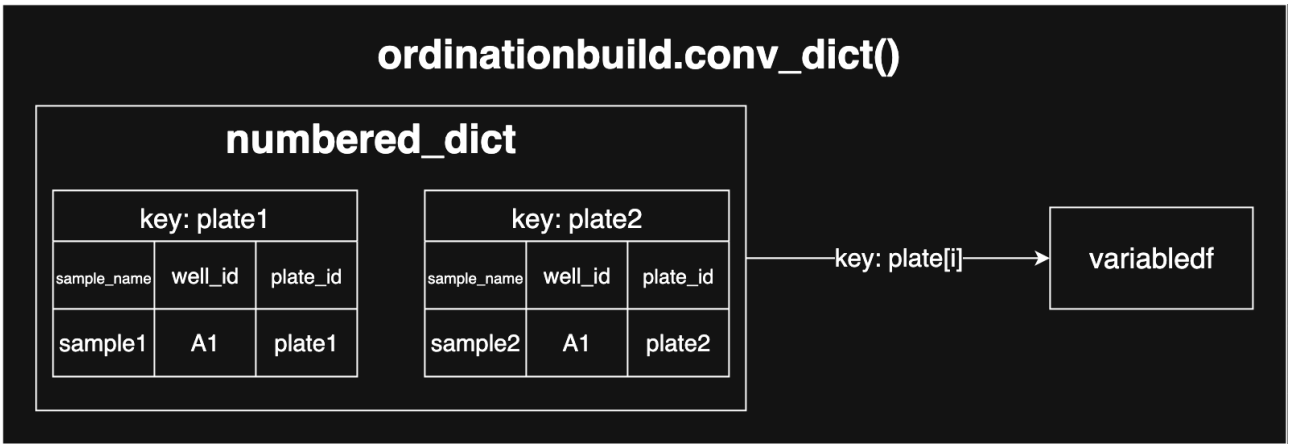

Figure 3: detailed description of ordinaitionbuild.conv_dict()

The DataFrame then undergoes heavy modification to be usable for scikit-bio's ordination functions. First, *ordinationbuild.ordinationbuild()* does scan the column *"well_id"* for availability, then it searches for any NaN values and deletes them. These NaN's exist because there can be more rows of sample names in column "*sample_name*" then well IDs in column "*well_id*" which is to be expected. By taking this step, platemapper ensures to only use real and expected well IDs. After this initial check, the well IDs get split up from one column into 2 separate columns, one containing the y-Axis values (e.g A-H), the other containing the x-Axis values (e.g 1-12) of one well ID. The well ID gets handled as a pandas Series, which splits the string in between the first and second character. The next modification consists of converting the row indication letters of a well ID (e.g. A-H) into numbers for placing the samples in a coordinated way. This gets realized by taking the letter of one well ID and comparing it with the indices of one specific string of letters: "HGFEDCBA". The reverse order of the letters is detrimental because unlike a 96 well plate which has its first well "A1" located in the top left corner, Emperors first coordinate "1,1" would be located at the bottom left. By assigning the numbers in reverse order, emperors coordination of wells is ensured to be in correct order.

The final steps consist of assigning the variables to the correct type, in case of skbio float or float64, and switching the position of the "row" and "column" values within the DataFrame because skbio needs the x-Axis values to be first in order to output a correctly oriented ordination.

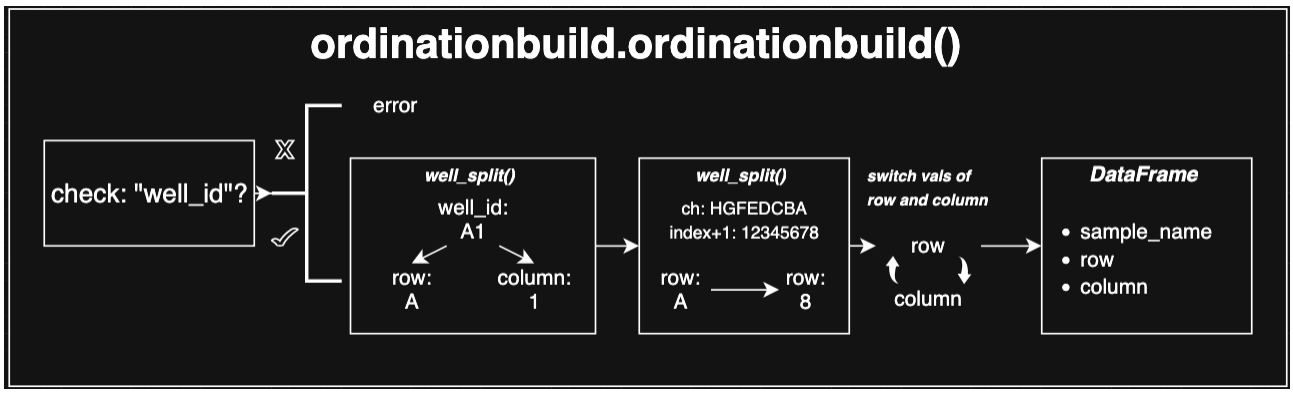

Figure 4: function ordinationbuild.ordinationbuild() simplified

The final DataFrame before the ordination gets created in *ordinationbuild.finalDataframeBuild()*, although it could be moved into the *ordinationbuild()* function. The final DataFrame consists of only 3 columns: The first being “sample_name”, the second being “row” which contains x-Axis values and the last one being “column” containing y-Axis values.

Afterwards, platemapper begins with creating an ordination file through *skbio.stats.ordination*. First, the ordination requires values for two positional arguments; “Eigenvalues” and “proportion_explained”. These values are quite important for a regular PCoA ordination as they define variance for singles axes or all axes combined. For platemapper, these values hold no meaningful value and will not be part of any equation of regular PCoA, that’s why platemapper uses made up values for ordination, as they are still required for ordination building. For ordination building, skbio demands both Eigenvalues and proportion explained, as well as a table containing sample names and Coordinates, these need to have the same number of dimensions as the Eigenvalues and proportion_explained, otherwise *OrdinationResults* throws an error. Lastly, platemapper saves the ordination from a variable into a .txt file for further visual processing from tools like *qiime2.Artifact, qiime2.Metadata* and *qiime2.plugins.emperor.visualizers.plot*. This ordination.txt file gets saved under platemapper/output/output_ordin.

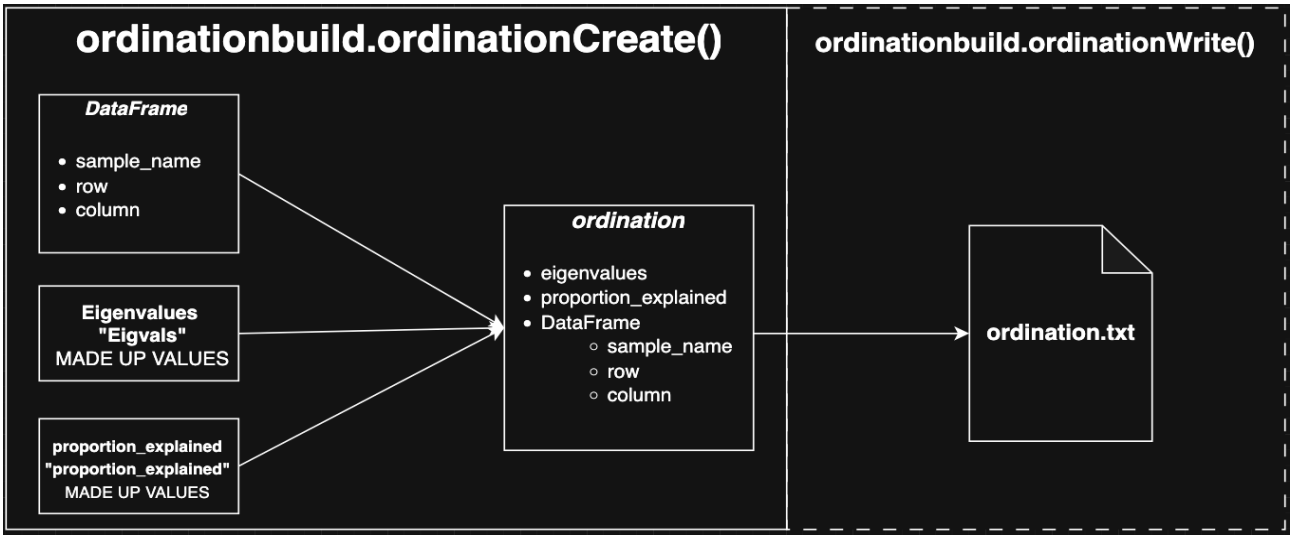

Figure 5: Description of .ordinationCreate() and .ordinationWrite()

After ordination building via ordinationbuild.py has been completed, platemapper does behave differently based on the number of processed plates. If there's only one plate present in the metadata table, platemapper takes that one ordination and directly builds an emperor compatible visualization of analyzed plate. If there's two or more plates found, platemapper combines all of them into one big ordination by adding the following plates with an offset into the ordination. This process is handled by main.py, where all ordinations get copied onto the variable *endplate.* After the first iteration, right before the well IDs get split into y- and x-Axis, the values of the x-Axis get offset by 13 length units, leaving one column space in between both plates. This "modified" plate then gets added into *endplate*, where the first plate already got copied in. Even with this offset, platemapper retains the original well ID positions in the metadata file, preventing giving wrong or modified results to the user. This process of adding plates into the table *endplate* happens based on the number of plates given by the metadata file. After processing all the plates, *endplate* gets build into one big ordination containing all plates.

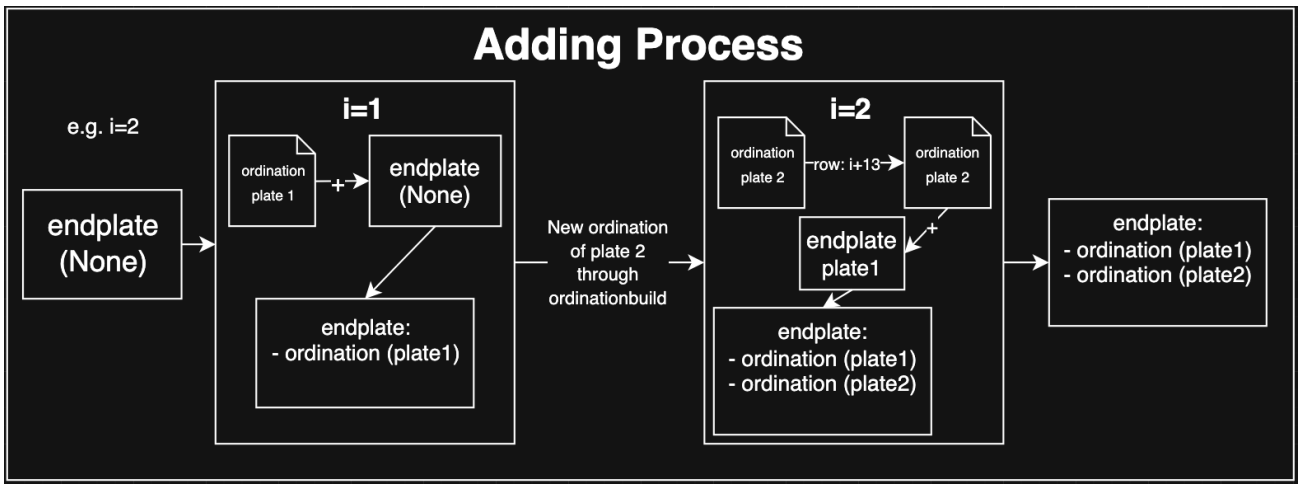

Figure 6: simplified showing of the adding process of multiple plates via endplate

Afterwards, platemapper begins with creating the desired visualization, all being handled by *qiimebuild.py*. An Emperor plot requires two things: a *QIIME Zipped Artifact* (.qza) file and the metadata file previously used for ordination building. A .qza file contains PCoA results as well as metadata and further information regarding file creation date etc. In order to get a .qza file, the function *qiimebuild.qzabuildsingle()* gets used. It takes the previously created ordination.txt and opens it with *qiime2.Artifact* with certain arguments like *view_type* (in this case *OrdinationFormat*) and saves it into a .qza file, which gets saved under platemapper/output/artifact_results.

For building an emperor-based visualization, the function *qiimebuild.empbuildsingle()* gets used to form the users data into the final visualization. It firstly loads both the .qza file and the initial metadata file. The metadata file provides all the other characteristics by which samples can be filtered which is quite useful as emperor already provides the user with all the tools to filter samples by characteristics. Then it builds the variable *vis* with *qiime2.plugins.emperor.visualizations.plot*, requiring arguments for the Artifact and metadata file, as well as the argument *ignore_missing_samples*=True, otherwise emperor cannot build visualizations if the metadata file contains samples without coordinates which can happen, but often this is intentionally made by the user. Variable *vis* then gets saved as a .qzv file, the file type which can be understood by emperor, into platemapper/output/emp_results. After successful creation of the visualization, both other folders *output_ordin* and *artifact_results* get emptied because as both files are only used to get the .qzv file and are therefore not longer of use.